

LEXIA: An AI-Powered Accessibility Tool for People with Dyslexia

Project Proposal for UCS503P
Submitted to: Mrs. NISHA THAKUR
Thapar Institute of Engineering and Technology

Nandini Sharma, CSED* Divyadeep Singh, CSED†

August 21, 2026

1 Higher-order goal

... secondary framing

This project aims to advance digital accessibility and educational equity by mitigating reading fatigue and comprehension barriers for individuals experiencing dyslexia and visual reading difficulties. Rather than attempting clinical medical diagnosis, the immediate engineering objective is to build a responsive, deployable web application that transforms complex text into an adaptive, distraction-free, and multimodal reading environment.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by prototyping the core text simplification and customizable typography reader within a rapid initial iteration.
- We will prioritize fast feedback over perfection by conducting structured usability trials and deploying early via continuous integration (CI) automation.
- Success is defined by end-to-end usability metrics (reading comprehension speed, task completion, interface clarity) achieved in the initial prototype and systematically refined through iterative testing.

3 Problem Statement

Individuals with dyslexia and related reading challenges encounter severe cognitive overload and fatigue when consuming dense digital text (e.g., academic articles, news, long-form documentation). Key pain points include:

- **Visual crowding and tracking fatigue:** Standard web fonts, tight line heights, and unyielding paragraph layouts cause eye-tracking loss and visual distortion.
- **Vocabulary and syntactical overload:** Dense clauses and polysyllabic words impede comprehension without quick, contextual clarification.

*Roll No: 1024170445, Email: nsharma7_be24thapar.edu

†Roll No: 1024170435, Email: dsingh_be24thapar.edu

- **High cognitive friction in assistive tooling:** Existing assistive tools are either paid desktop extensions, ad-cluttered readers, or inflexible screen readers lacking text simplification.
- **Lack of personalized pacing:** Readers cannot easily switch between varying degrees of reading assistance (e.g., sentence-by-sentence focus vs. key-point summaries).

Why this matters now (contextual relevance): As academic and professional environments increasingly rely on digital content, lightweight and accessible assistive tools are critical to preventing educational disadvantages and cognitive burnout.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based** accessibility reading platform (**LEXIA**) composed of:

- **Adaptive Reading Interface:** Highly customizable viewer supporting dyslexia-friendly fonts (e.g., OpenDyslexic, Lexend), dynamic letter/line spacing, background tint adjustments, and reading rulers.
- **Multi-Tier AI Simplifier:** Backend pipeline utilizing Large Language Model APIs (Google Gemini) to deliver 3 adaptive simplification levels (Basic, Easier, Very Simple) while preserving core meaning.
- **Contextual Explainer & Summarizer:** On-demand extraction of key takeaways and instant, context-aware definitions for difficult vocabulary.
- **Multimodal Assistance:** In-browser text-to-speech (TTS) synchronized with visual sentence tracking and a distraction-free “Focus Mode”.
- **User Personalization & Library:** Authenticated profile managing persistent typographic settings, saved articles, and reading history.

4.2 Core workflow

1. User pastes text or selects a sample document into the LEXIA portal.
2. Backend parses content and triggers the requested transformation (Simplification, Summary, or Word Explanation) via controlled API prompts.
3. React frontend renders adapted content in the reading interface according to the user’s saved typography preferences.
4. User engages interactive tools (Focus Mode, Read Aloud, Word Lookup) and optionally saves the adapted material to their library.

4.3 Operational constraints for an educational, web setting

- Keep the interface lightweight and responsive across desktop and tablet browsers without bulky browser extensions.
- Avoid local model training overhead; leverage production-ready LLM APIs with strict prompt engineering and schema validation.
- Implement defensive timeout handling and local fallbacks for seamless evaluation and offline reader functionality.

5 Solution Approach

... Engineering focus

This project focuses on reliable web engineering, API orchestration, and accessibility UX compliance rather than training custom machine learning models from scratch.

5.1 AI Orchestration & Safety Layer

... non-research

The AI service is decoupled into specialized stateless controller routines:

- **Controlled Prompting:** Strict system prompts enforce deterministic vocabulary reduction, sentence splitting, and factual preservation without extraneous hallucinations.
- **Contextual Word Disambiguation:** Sends selected tokens alongside surrounding sentence context to ensure accurate definitions.
- **Defensive Validation:** Validates JSON responses and enforces character limits to guarantee fast response times and avoid rate-limiting bottlenecks.

5.2 Web architecture

... web-based solution

A decoupled 3-tier web architecture:

- **Frontend (React + Tailwind CSS):** Modular UI managing reading state, Web Speech API synthesis, custom typography engines, and local preference caching.
- **Backend API (Django REST Framework):** Stateless endpoints managing user auth, database persistence, and AI controller orchestration.
- **Database (SQLite / PostgreSQL):** Relational storage maintaining user credentials, accessibility preference profiles, articles, and versioned AI results.

5.3 CI/CD and fast delivery enabler

CI/CD pipelines will be configured to:

- Run automated unit and integration tests (Pytest for backend, React Testing Library for frontend).
- Enforce code formatting, linting, and accessibility audit checks (axe-core / Lighthouse).
- Deploy containerized/hosted builds to staging environments (e.g., Vercel + Render).

6 Evaluation Criterion

... measurable and attributable

We evaluate success using measurable performance and user-centric accessibility metrics.

6.1 Primary evaluation metric

End-to-End Processing Latency (EPL): Time elapsed from user submission to rendered simplified text in the reading interface.

- Target: Median EPL ≤ 3.0 seconds for standard articles (≤ 800 words).
- Attribution: Measured via server-side response logging and frontend performance timestamps.

6.2 Secondary evaluation metrics

- **Readability Improvement:** Quantified drop in Flesch-Kincaid Grade Level between source text and simplified output.
- **Comprehension & Reading Speed:** Pilot testing measuring time taken to read and answer basic comprehension queries.
- **Interface Usability Score:** User satisfaction assessed via a standard System Usability Scale (SUS ≥ 80 target).
- **System Availability:** Uptime $\geq 99\%$ during pilot testing windows.

6.3 Pilot validation plan

... high-level

- Recruit 10–15 student volunteers with varying reading proficiencies to execute standardized reading tasks across original vs. LEXIA-adapted texts.
- Gather quantitative feedback on font efficacy, focus mode tracking, and vocabulary clarity through pre- and post-test surveys.

7 Scalability

... with foresight

The application is designed for horizontal scaling and graceful degradation:

1. **Scalable (theoretically):** Decoupled REST architecture allows client-side caching of user preferences and static assets, reducing database overhead. Asynchronous AI request queues prevent blocking web workers during traffic spikes.
2. **Operationally deployable (practically):** Container-ready services deployable on managed PaaS tiers with database connection pooling and environment-isolated secrets management.

8 Engine availability heuristic

Standard, production-grade tools and libraries will be utilized:

- **Frontend:** React.js, Tailwind CSS, Web Speech API.
- **Backend:** Python 3, Django, Django REST Framework.
- **AI/LLM Engine:** Google Gemini API with JSON schema enforcement.
- **Persistence:** PostgreSQL / SQLite with Django ORM.
- **Testing & CI:** Pytest, React Testing Library, GitHub Actions CI runner.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Core text input portal with live paste functionality.
- Functional reading interface featuring dyslexia-friendly fonts, font resizing, and line-spacing controls.

- AI simplification pipeline (single tier) integrated with backend API.
- Basic Web Speech API read-aloud functionality.
- CI pipeline with unit test verification on every commit.

9.2 Subsequent deliverables

- Multi-tier simplification selector (Basic, Easier, Very Simple).
- Contextual word explanation popup on text highlight.
- Focus Mode (sentence-by-sentence and line-mask reading).
- User authentication, persistent preference profiles, and saved article history.
- Automated readability scoring dashboard.

10 Risks and mitigations

- **AI Hallucination / Distortion:** Enforce conservative prompts with explicit negative constraints (“Do not add external facts”); provide split-view comparison with original text.
- **API Latency & Quotas:** Implement client-side loading skeletons and backend response caching for repeated queries.
- **Browser Inconsistencies in Speech Synthesis:** Utilize standard Web Speech API with fallback controls and clear UI status indicators.
- **Accessibility Compliance Gaps:** Continuously validate UI elements against WCAG 2.1 AA standards using automated linters and keyboard-navigation tests.

11 Summary

LEXIA is a web-based, AI-augmented accessibility reading platform designed to reduce visual strain and cognitive load for individuals with reading difficulties. Aligning with course objectives, the project prioritizes rapid time-to-value, automated testing via CI/CD, and measurable evaluation metrics (processing latency, readability enhancement, and usability scores) within a robust engineering architecture.